

Start

ARRAYS # (Most Important) Part - 1

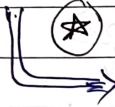
Why? ⇒ Whenever you want to store any data/any information we create variables. If we want more data we have to create more variables with different name. If we want ~~to~~ store 100 student roll number it's not possible to create 100 different variables. So, then ~~we~~ Array comes into picture.

e.g:- 100 Roll no →

```
int a; or int a, b, c, d, e, ... aa, ... aab, aac ...  
int b;  
int c;  
int ab;  
int aab;
```

→ `int roll = {1, 2, 3, 4, 5, ..., 99, 100};`

What?



So, What is Array?

An array is a homogeneous non-primitive data type used to save multiple elements (having the same data type) in a particular variable.

So, Arrays in Java can hold primitive data types (Integer, Character, Float etc) and Non-primitive data type (Objects, Strings)

⇒ Array = ⁽ⁱ⁾ Index based Data Structures to store large volume of Data using single variable name.

(i) Homogeneous data stores only

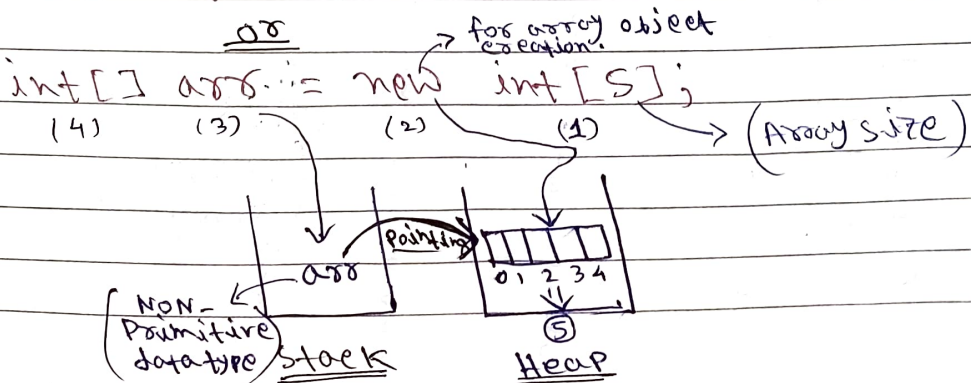
(ii) Arrays in Java is treated as Objects → that's why the memory is allocated in Heap Area.

this is why we need new keyword to create array.

Short Definition:- Array is a index based collection of same type of data.

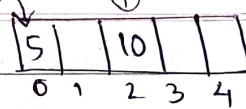
(*) Syntax of array:-

```
int[] arr = {1, 2, 3, 4, 5, ...};
```



arr[2] = 10;

arr[0] = 5;



How?

⇒ Cases: - (example)

1D ARRAY
(regular)

★

2D ARRAY
(regular)

★

3D ARRAY
(regular)

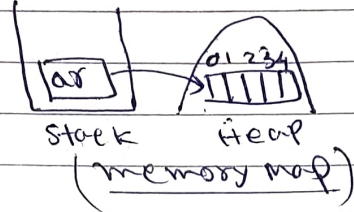
★

① To store marks of 5 students →

⇒ Student ⇒ `int[] ar = new int[5];`

Syntax of 1D Array

1D Array

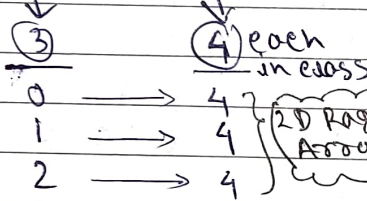


② To store marks of 3 classes each with 4 students →

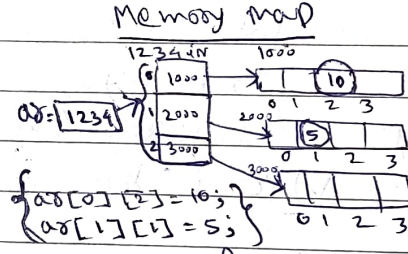
⇒ (classes) (students) ⇒

Syntax of 2D -

`int[][] ar = new int[3][4];`



2D Array this is



NOTE:- there are two types of array in java.

by type → (a) Regular array, (b) Jagged array.

a) Regular ⇒ Same type of data stored. e.g. `[4]`

b) Jagged ⇒ different or irregular type of data stores. e.g. `[5]`

③ To store marks of 3 colleges, each college's 4 classes & class's 3 students each

#3D

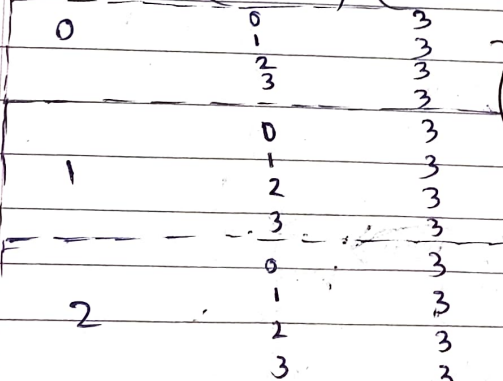
(colleges) (classes) (students) ⇒

Syntax 3D ⇒

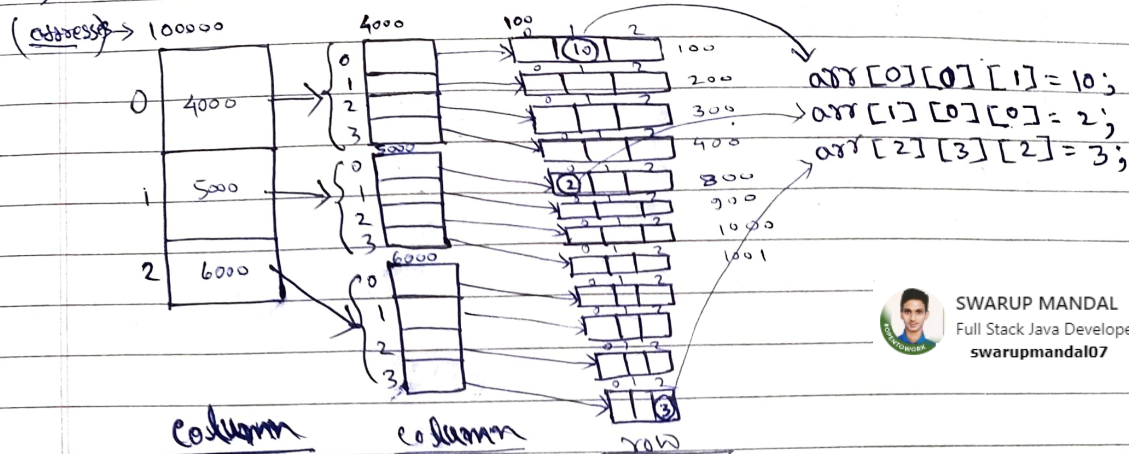
`int[][][] ar = new int`

[3] [4] [3]
↓ ↓ ↓
column column row

#3D Regular array

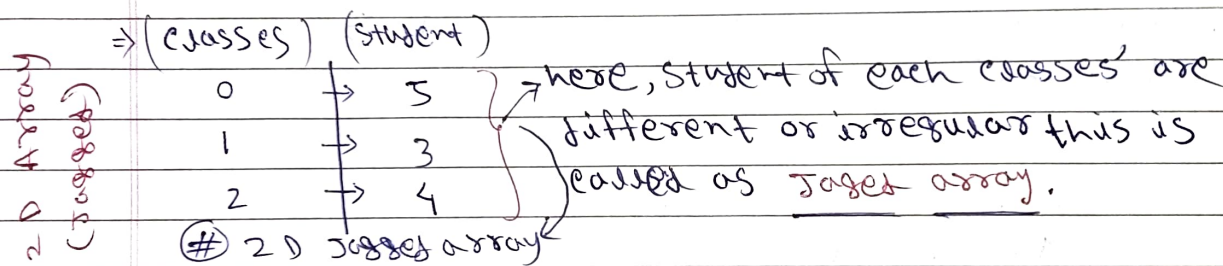


⇒ memory map of 3D Array:—



SWARUP MANDAL
Full Stack Java Developer
swarupmandal07

④ To store marks of 3 classes with 5, 3, 4 Student respectively of classes:—



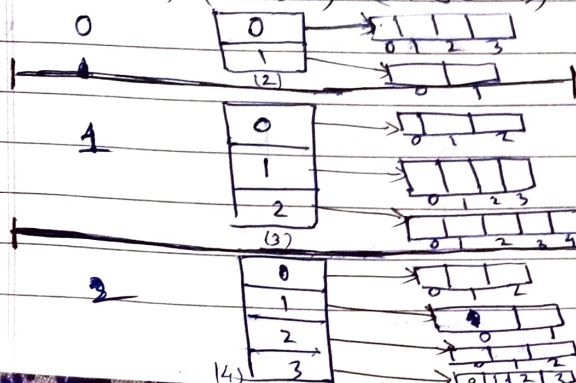
⇒ Syntax of Jagged array:—

```
int[][] arr = new int[3][ ];  
arr[0] = new int[5];  
arr[1] = new int[3];  
arr[2] = new int[4];
```

must give for at least for 1D array
leave as blank (row)

⑤ To store marks of (3) colleges, each colleges (2, 3, 4) classes respectively & class's [4, 2] [3, 4, 5] [3, 2, 3, 4] Students →

⇒ (colleges) (classes) (Students)



```

{
ar[1][0] = new int [3];
ar[1][1] = new int [4];
ar[1][2] = new int [5];
}
{
ar[2][0] = new int [3];
ar[2][1] = new int [2];
ar[2][2] = new int [3];
ar[2][3] = new int [4];
}

```

blank (for diff)

⇒ if `int[][][] arr = new int [3][2][ ]`

```

arr[0][0] = new int [3];
arr[0][1] = new int [4];
arr[1][0] = new int [2];
arr[1][0] = new int [4];
arr[2][0] = new int [3];
arr[2][1] = new int [4];

```

3x2 = 6 row

initialization

```

full
arr[0][0][0] = 10;
arr[1][2][0] = 30;

```

Multi Dimensional Array:-

⇒ 1D, 2D, 3D ⇒ Regular

2D, 3D ⇒ Jagged

4D

#code snippets

```

1) while (1) { // 1 is not boolean in java
    S.O.PR("Hello");
} } }

```

Ans:- C.E

Exception Error

2) while (true) ⇒ it gives ~~infinite loop~~

3) ~~while (true)~~

S.O.PR("swamp"); // single stmt-1 after while

⇒ It gives infinite loop of continuously printing swamp as output. you have to forcefully stop it

```

4) while (true) {
}

```

⇒ It gives NO-output.

5) while (true); // It gives NO-output (is getting of any statement)

6) while (true); ⇒ it gives C.E

S.O.PR("Swamp");

Syntax:-

```
while (boolean) {
```

```
    // stmt-1
    // stmt-2
```

```
} // without initialization and increment/decement while can run
```

Q) while (true) $\Rightarrow$ It gives C.E (bez declarative stmt not allowed in any loop) if you use without $\{ \}$ put in between $\{ \}$ it is valid to declaration.

Q) while (true) {
int x = 10;
}

Q) while (true) {
S.o. Print("hello"); // Line-1 $\checkmark$
}
S.o. Print("hee"); // Line-2 $\checkmark$
}

$\Rightarrow$ it gives compile time error at line-n2 # (unreachable code) to line-n2
 $\rightarrow$ This statement will continuously run (infinite)

Q) while (false) {
S.o. Print("hello"); // if you make it false why you write stmt-
}
S.o. Print("hee");
}

$\Rightarrow$ it gives compile time error at line-n1

Q) int a = 10, b = 20;
while (a < b) {
S.o. Print("hee");
}
S.o. Print("hello");
}

$\Rightarrow$ It gives hee as a infinite output bez (a < b = true) $\Rightarrow 10 < 20 = \text{true}$
 $\Rightarrow$ variable comes with value and doing relational operation $\Rightarrow$ result will be boolean. if true print (any thing you want)

Q) final a = 10, b = 20; // whenever variables are marked as final, compiler will get to know the value of those variables and it will use the values in the expression to get result.
while (a < b) {
S.o. Print("a");
}
S.o. Print("b");
}

$\Rightarrow$ This expression will evaluate by compiler
// (unreachable code) to (line-2) bez line 1 give infinite loop

Q) final a = 10;
while (a < 20) {
S.o. Print("a");
}
S.o. Print("b"); // Line #2

$\Rightarrow$ # evaluated by compiler (compiler knows) $\Rightarrow$ It gives C.E at line #2

Q) do
S.o. Print("hello"); // line-n1
while (true);
}

$\Rightarrow$ do print the value and while (true) print infinite the statement of do. at least once

Q) do;
while (true);
Ans: No output

Q) do
int x = 11; // line 1
while (true);
Ans: C.E at line-1

Q) do {
int x = 10;
} while (true);
Ans: No output ($\checkmark$)

P.S. vm(SEC0083)

I. Q) `do while(true); // P.E (because) inside while loop is not ends with ;`
`System.out.println("hello"); // starts` ^{otherwise it will print infinite true starts}
`while(true);`
`}`
`}`

So, What compiler will do:-

do

`while(true);` → ~~is not~~ have to remove

`System.out.println("hello");`

`while(true);`



Array # (continued)

Part - 2

classmate

Date 28/10/2022
Page

*** User input from console in java: -

⇒ To take input from user we have to import a class called **Scanner** which is present in **Util** library. ^{Package}

⇒ **Scanner** $\Rightarrow$ **Methods** in Scanner class there have so many method. like - **nextInt**, **nextFloat**, etc.

(is a class) $\swarrow$ that have many $\searrow$ to take those method we have to create a object of scanner class

(present in Util Package) $\swarrow$ **java.util.Scanner**

create (object)

★ Rules to taking input from user/console: -

(i) you have to import a package **import java.util.Scanner**

(ii) To use the Scanner class's methods you have to create object of that class to give life and also pass a taking input stream (**System.in**) which connect to the input service.

Scanner scan = new Scanner(System.in);



$\downarrow$ set variable $\downarrow$ keyword

input stream
System $\Rightarrow$ final class
in $\Rightarrow$ instance (variable)
out $\Rightarrow$ instance of PrintStream class

(iii) Now, after creating object you can call all methods of Scanner class by the **reference variable** you created and store it in a local variable as per data type.

iii/a) To take integer input $\rightarrow$

int sm = Scan.nextInt();
 $\swarrow$ (data type) $\swarrow$ (l.v) $\swarrow$ (r.v) $\swarrow$ (method)

b) To take float input $\rightarrow$

float sm = scan.nextFloat();

c) To take String input $\rightarrow$

String name = scan.next(); $\rightarrow$ to take input before space
String name = scan.nextLine(); $\rightarrow$ To take input whose line increasing space

d) **nextDouble();** $\Rightarrow$ to take double input.

e) **nextByte();** $\Rightarrow$ " " byte "

f) **nextBoolean();** $\Rightarrow$ " " Boolean "

g) **nextLong();** $\Rightarrow$ " " Long "

h) **nextShort();** $\Rightarrow$ " " Short "

i) **BigInteger se = Scan.nextBigInteger();** $\Rightarrow$ input as a BigInteger $\rightarrow$ long ^(import java.math.BigInteger)

j) **BigDecimal se = Scan.nextBigDecimal();** $\Rightarrow$ take input as a $\rightarrow$ double ^{BigDecimal}

k) **next().charAt(0);**
nextLine().charAt(0); $\rightarrow$ (import java.math.BigDecimal;) $\rightarrow$ double

1 D Array taking input (Regular)

⇒ Want to take marks from user and store it in array and want to display 5 student's marks.

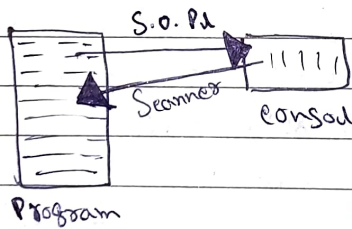
```
import java.util.*;
public class Inputs {
    P.S.V.M (S.E) a {
        int[] arr = new int[5];
        Scanner scan = new Scanner(System.in);
        // arr[0] = new scan.nextInt();
        // arr[1] = scan.nextInt(); ..... till arr[4] (NO) X
```

taking input by for loop

```
        for (int i=0; i<5; i++) {
            S.O.Pln("Enter marks of student: " + i);
            arr[i] = scan.nextInt();
        }
        // Now want to display the taking input.
```

Display input by for loop

```
        for (int i=0; i<5; i++) {
            S.O.Pln(arr[i] + " ");
        }
```



Array last index = 6
 Array Length = 6 + 1 = 7
 index → 0 1 2 3 4 5 6

⇒ Length ⇒ it is a property of array. length is a variable. its duty is get the length of the array.

• Syntax: - Array Length = (Array's Last Index + 1)
 array's variable_name . length

• ex:- arr.length;

```
⇒ int arr = new int[6];
   for (int i=0; i<arr.length; i++) {
       arr[i] = scan.nextInt();
   }
```

⇒ Scanner scan = new Scanner(System.in);
 S.O.Pln("Enter the number of student by user:");
 int size = scan.nextInt();
 int[] arr = new int[size];



2D Array taking input (Regular)

```
import java.util.*;
```

```
public class Inputs2d {
```

```
    P.S.V.M (Scanner args) {
```

```
        int [][] arr = new int [3][4]; // array declaration.
```

```
        Scanner scan = new Scanner (System.in);
```

```
        for (int i=0; i<arr.length; i++) {
```

```
            for (int j=0; j<arr[i].length; j++) {
```

```
                S.o.pl ("Enter marks of class" + i + "student" + j);
```

```
                arr[i][j] = scan.nextInt();
```

```
            }
```

```
        }
```

```
    }
```

What happens?

$\therefore arr.length = [3]$
(i)

$\Rightarrow i = 0; 0 < 3 \Rightarrow$
(i)

$j = 0; 0 < 4 \Rightarrow$
(j)

$\therefore arr[i].length$

$\Rightarrow arr[0].length \Rightarrow [4]$
(j)

arr (0) → [] [] [] [] (4)
1 → [] [] [] [] (4)
2 → [] [] [] [] (4)

$arr[i][j] = scan.nextInt();$

$arr[0][0] = 3$

$j++ \Rightarrow 0 \rightarrow 1$

$arr[0][1] = 4$

$arr[0][2] = 5$

$arr[0][3] = 3$

$\rightarrow arr[0][3] \rightarrow$

$j++ \Rightarrow 0 > 1$

then again doing same

```
        for (int i=0; i<arr.length; i++) {
```

```
            for (int j=0; j<arr[i].length; j++) {
```

```
                S.o.pl ("marks of class" + i + "student" + j);
```

```
                S.o.pl (arr[i][j] + " ");
```

```
            }
```

```
        }
```

```
    }
```



SWARUP MANDAL
Full Stack Java Developer
swarupmandal07

2D Jagged array taking input

```
int [][] arr = new int [3][ ];
```

```
arr[0] = new int [3];
```

```
arr[1] = new int [2];
```

```
arr[2] = new int [4];
```

(3)

(i) → 0 1 2 = (3)
0 → [] [] []
1 → [] [] = (2)
2 → 0 1 2 3 = (4)
[] [] [] []

```
Scanner scan = new Scanner (System.in);
```

```
# for ( ) // same for-loop for taking input (no change of
```

```
# for ( ) // same for-loop for display input ( " " )
```

$arr[0].length = (3)$

$arr[1].length = (2)$

$arr[2].length = (4)$

Automatically take

blank (How much row
student is in
know)

3D Regular Array taking input: -

colleges (i)	class (j)	students (k)
0	0	3
	1	3
	2	3
1	0	3
	1	3
	2	3

⇒ import java.util.*;

public class Inputs {

public static void main(String args[]) {

Scanner sony = new Scanner(System.in);

int[][][] arr = new int[2][3][3];

① for (int i = 0; i < arr.length; i++) {

② for (int j = 0; j < arr[i].length; j++) {

③ for (int k = 0; k < arr[i][j].length; k++) {

S.o.println("Enter marks of colleges " + i + " class

+ j + " student" + k);

arr[i][j][k] = sony.nextInt();

}

S.o.println("Printing your typed input");

for (int i = 0; i < arr.length; i++) {

for (int j = 0; j < arr[i].length; j++) {

for (int k = 0; k < arr[i][j].length; k++) {

S.o.println("marks of colleges " + i + " class" + j +

S.o.println(arr[i][j][k] + " "); "student" + k);

}

}

}

}

3D Jagged Array taking input: -

Scanner scan = new Scanner();

int[][][] arr = new int[2][][3];

{ int [0] = new int[2][3];

int [1] = new int[1][3];

⇒ int [0][0] = new int[3];

int [0][1] = new int[4];

⇒ int [1][0] = new int[5];

for (i) {

for (j) {

for (k) {

}

}

}

}

// Same as above from for loop.



3) How Array data is secure in Ram?

⇒ Ram is a collection of bytes. So, when we create an array suppose (`arr[2]`) then we know it is 4 bytes. So, $2 \times 4 = 8$ bytes will be occupied in ram by those array. When we initialize our array by values of every index then all the bytes area will filled. If we filled extra indexes with value then it is a Buffer overflow happened. → `arr[2] ⇒ arr[0] = 10; ①`
`arr[1] = 20; ②`

`arr` →

0	1	2	3	4	5	6	7	8	9

 // extra → `arr[3] = 40; ③` (Buffer overflow)
x
Buffer (2 bytes) overflow (2 bytes)

⇒ Buffer overflow / Buffer overflow :-

A buffer overflow (or buffer overrun) occurs when the volume of data exceeds the storage capacity of the memory buffer which user chooses. It will be allowed for ~~extra~~ write, it does not show any error. When we write it will show in compile-time. Buffer overflow can see on C, C++ programming.

⇒ Buffer overflow Attack ? :-

Attackers exploit buffer overflow issues by overwriting the memory of an application. This changes the execution path of the program, triggering a response that damages files or exposes private information.

• Example:- an attacker may introduce extra code, sending new instructions to the application to gain access to IT systems. If attackers know the memory layout of a program, they can intentionally feed input that the buffer cannot store, and overwrite areas that hold executable code, replacing it with their own code. For example, an attacker can overwrite a pointer (an object that points to another area in memory) and point it to an exploit payload, to gain control over the program.

⇒ Type of Buffer overflow Attacks:-

a) Stack-Based buffer overflow b) Heap-Based attacks

⇒ How to prevent Buffer overflows:-

Developers can protect against buff. ove. Vulnerabilities via security measures in their code, or by using languages that offer built-in protection.

→ In addition, modern operating system have runtime protection. Three common protection are →

a) address space randomization (ASLR)

b) Data execution prevention

c) Structured exception handler overwrite protection (SEHOP)

d) The Improve security solution:-

(i) DDoS protection, (ii) web application firewall,

(iii) Bot management, (iv) Account takeover protection.

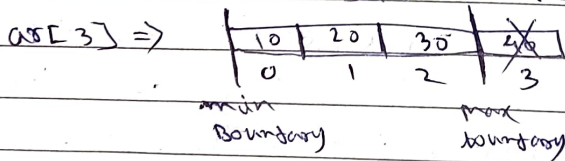
(v) API security (vi) RASP, (vii) Attack analytics.

8) Does in java buffer overflow happens?

⇒ NO, Language such as PERL, javascript, C++ and java use built-in safety mechanisms that minimize the likelihood of buffer overflow.

In other words, data (array) is bounded in java by Boundary → min. Boundary and max Boundary. If you write extra data for a location it gives.

Runtime error (Exception) → ArrayIndexOutOfBoundsException and the code ~~will~~ terminated.



⇒ So, Array is Guarded with Boundaries to protect from Buffer overflow/under attacks.

⇒ We can't take input as a character only. But we can create character arrays no issue.

⇒ We can store or create any type of array → int, char, float, double, long, String, ...



⑤

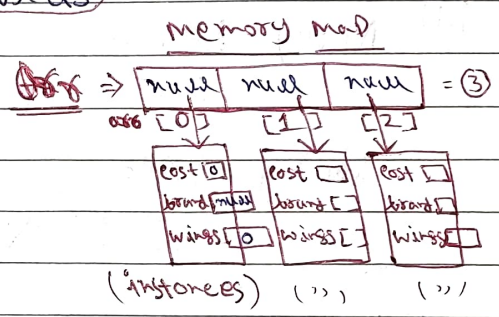
④ Creating a object's (including class) ~~Array~~ Array: -

```
class Fan {
    int cost; // instance variables / Field / property
    String brandName;
    int noOfWings;
}
```

```
public class Array {
    P. S. V. M (Set array) {
```

Fan[] arr = new Fan[3]; // so, we can create array of
// creating Fan class array & objects every types / things)

```
{
    arr[0] = new Fan();
    arr[1] = new Fan();
    arr[2] = new Fan();
}
```



```
{
    arr[0].cost = "1000";
    arr[0].brandName = "USHA";
    arr[0].noOfWings = 3;
    arr[1].cost = 1100;
    arr[1].brandName = "Havells";
}
```

```
} }
```



Disadvantages of an array:-

i) It can store only homogenous type data.

```
int a[] = new int [4];
```

```
a[0] = 10; ✓
```

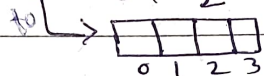
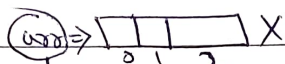
```
a[1] = 10.5f; X
```

```
a[2] = "jason"; X
```



ii) memory of Array is fixed in size (it cannot grow or shrink)

```
int a[] = new int [3];
```



iii) Array demands contiguous memory locations:-

```
int a[] = new int [2];
```

```
int => 4 * 2 = (8) Bytes
```

are required
if occupied by
others that's
not possible
for array



Bytes

(Array requires contiguous free locations to store value.)

If in the middle others
interruption not accepted.

So, To overcome array we have ArrayList.

Recommended syntax of array:-

```
int[] a = new int [2];
```

You can use `a[]` or `a[ ]` does not matter but recommended is `a[]`. because it is more readable by the user/developer.

```
=> int a[] = new int [3]; // valid (But not Recommended)
```

int type
1D array
of

means $\Rightarrow$ `int[] a = new int [2];` // valid (Recommended)
[a is 1D Array of int type data type]

We can directly assign value to the Array

① 1D Array: `int[] a = {10, 20, 30, 40, 50};`

② 2D Regular Array: `int[][] a = {{10, 20}, {20, 30}};`

2 student each

crosses

- 3) 2D Jagged:- $\text{int}[][] a = \{ \{10, 20\}, \{30, 40, 50\} \}$
 Diagram: $\{10, 20\}$ is labeled "2 students of 0 class". $\{30, 40, 50\}$ is labeled "3 students of 1 class". The entire array is labeled "2 classes".
- 4) 3D Regular:- $\text{int}[][][] a = \{ \{ \{10, 20\}, \{20, 30\} \}, \{ \{30, 40\}, \{50, 60\} \} \}$
 Diagram: $\{10, 20\}$ and $\{20, 30\}$ are labeled "students". $\{30, 40\}$ and $\{50, 60\}$ are labeled "courses". The entire array is labeled "classes each & 2 students each".
- 5) 3D Jagged:- $\text{int}[][][] a = \{ \{ \{10, 20, 30\}, \{10, 30\} \}, \{ \{40, 50\}, \{1, 2, 3, 4\} \} \}$
 Diagram: The innermost arrays are labeled "3 student in a college or class".

6) We can write:- $\text{int}[] a = \text{new int}[] \{10, 20, 30, 40\};$
 this is also possible for 1D, 2D, 3D array.

(Enhanced)

★ For-each loop:-

For-each is another array traversing technique like for loop, while loop, do-while loop introduced in Java 5.

⇒ Syntax:- $\text{for}(\text{type } \text{variable} : \text{array name})$
 $\{$
 Statement using variable;
 $\}$
 Note: it can be (int, float, double) Start point (you don't add to do anything)

⇒ example:- (1D Array)

```
int[] a = {10, 20, 30, 40};
for (int element : a) {
    S.O. Print(element);
}
```

$a \rightarrow$

0	1	2	3
10	20	30	40

Go to first index extract the value and give to the variable (element) then print it again go to next index of that array and keep repeating.

⇒ Where? Whenever as a programmer, you are not aware or you don't want to give from which index you want to access from which index you want to go, and

the problem will go with the concept of For-each loop

⇒ Why? In java for-each loop is used to iterate through elements of arrays and collections (like linked list)

Advantages ⇒ i) it helps us avoid programming errors. ii) it makes the code precise and readable. iii) easier to implement. iv) avoids the chance of an infinite array.

Disadvantages ⇒ i) we can't jump over an element as it traverses through each element. ii) Traversing in reverse order not possible. iii) we can't modify array. iv) it's not possible to keep track of the index. v) it has some performance overhead over